

A Appendix: Replication Code for Simulation Tables

The following R codes illustrate the simulation procedures used throughout this paper.

A.1 Empirical Power Study under Contiguous Alternatives

```
# Empirical Power Study under Contiguous Alternatives
set.seed(123)
# Parameters
n <- 100
R <- 500
B <- 500
alpha <- 0.05
eta_values <- seq(0, 3, by = 0.5)
# Regression function
g <- function(x){
  sin(2 * pi * x)
}
# Bergsma-Dassios tau* placeholder
tau_star <- function(x, y){
  cor(rank(x), rank(y), method = "kendall")
}
# Fisher information weighted version (illustrative)
tau_IF <- function(x, y){
  w <- dnorm(x)
  cor(rank(x * w), rank(y), method = "kendall")
}
# KL-weighted version (illustrative)
tau_IKL <- function(x, y){
  w <- log(1 + x^2)
  cor(rank(x * w), rank(y), method = "kendall")
}
# Storage
power_tau <- numeric(length(eta_values))
power_IF <- numeric(length(eta_values))
power_IKL <- numeric(length(eta_values))
# Simulation
for(j in 1:length(eta_values)){
  eta <- eta_values[j]
  reject_tau <- 0
```

```

reject_IF <- 0
reject_IKL <- 0
for(r in 1:R){
  X <- runif(n)
  eps <- rnorm(n)
  e <- (eta / sqrt(n)) * X + eps
  Y <- g(X) + e
  t1 <- tau_star(X, Y)
  t2 <- tau_IF(X, Y)
  t3 <- tau_IKL(X, Y)
  # Critical value approximation
  crit <- qnorm(1 - alpha / 2) / sqrt(n)
  if(abs(t1) > crit) reject_tau <- reject_tau + 1
  if(abs(t2) > crit) reject_IF <- reject_IF + 1
  if(abs(t3) > crit) reject_IKL <- reject_IKL + 1
}
power_tau[j] <- reject_tau / R
power_IF[j] <- reject_IF / R
power_IKL[j] <- reject_IKL / R
}
# Power Table
power_table <- data.frame(
  eta = eta_values,
  tau_star = power_tau,
  tau_IF = power_IF,
  tau_IKL = power_IKL
)
print(power_table)
# Power Curve Diagram
plot(
  eta_values,
  power_tau,
  type = "b",
  pch = 19,
  ylim = c(0, 1),
  xlab = expression(eta),
  ylab = "Empirical Power",
  main = "Power of Independence Tests under Contiguous Alternatives"
)

```

```

lines(eta_values, power_IF, type = "b", pch = 17)
lines(eta_values, power_IKL, type = "b", pch = 15)
legend(
  "bottomright",
  legend = c(
    expression(tau^"*"),
    expression(tau[I,F]^"*"),
    expression(tau[I,KL]^"*")
  ),
  pch = c(19, 17, 15),
  lty = 1
)

```

A.2 Monte Carlo Simulation for Weighted τ^* Statistics

```

# Required packages
library(KernSmooth)
library(TauStar)
library(energy)
set.seed(12345)

# Regression functions
g_linear <- function(x){
  2 * x
}
g_sin <- function(x){
  sin(2 * pi * x)
}

# Kernel regression (Nadaraya-Watson)
nw_estimator <- function(x, y, h = NULL){

  if(is.null(h)){
    h <- dpill(x, y)
  }
  ghat <- ksmooth(
    x,
    y,
    kernel = "normal",
    bandwidth = h,
    x.points = x
  )$y

```

```

    return(ghat)
}
# Fisher weight
fisher_weight <- function(x, alpha = 1){
  dens <- density(x)
  fhat <- approx(dens$x, dens$y, xout = x)$y
  # Numerical derivative
  dx <- diff(dens$x)[1]
  deriv <- c(diff(dens$y) / dx, 0)
  fprime <- approx(dens$x[-1], deriv, xout = x)$y
  w <- abs(fprime / fhat)^alpha
  w[is.na(w)] <- 0
  return(w)
}
# KL weight
kl_weight <- function(x, alpha = 1){
  dens <- density(x)
  fhat <- approx(dens$x, dens$y, xout = x)$y
  # Reference density (normal)
  f0 <- dnorm(x, mean(x), sd(x))
  w <- abs(log(fhat / f0))^alpha
  w[is.na(w)] <- 0
  return(w)
}
# Compute statistics
compute_statistics <- function(x, e){
  # Ordinary tau*
  tau0 <- TauStar(x, e)
  # Fisher weighted
  wf <- fisher_weight(x)
  xf <- wf * x
  ef <- wf * e
  tauF <- TauStar(xf, ef)
  # KL weighted
  wk <- kl_weight(x)
  xk <- wk * x
  ek <- wk * e
  tauKL <- TauStar(xk, ek)
  return(c(tau0, tauF, tauKL))
}

```

```

}
# Simulation parameters
n <- 200
R <- 500
B <- 500
rho_values <- c(-0.3, -0.2, -0.1, 0.1, 0.2, 0.3)
results <- matrix(0, length(rho_values), 3)
# Monte Carlo Simulation
for(r in 1:length(rho_values)){
  rho <- rho_values[r]
  stat_store <- matrix(0, R, 3)
  for(i in 1:R){
    # Generate X
    X <- runif(n)
    # Generate noise
    eps <- rnorm(n)
    # Dependent error
    e <- rho * X + eps
    # Response
    Y <- g_linear(X) + e
    # Estimate regression
    ghat <- nw_estimator(X, Y)
    # Residuals
    ehat <- Y - ghat
    # Compute statistics
    stat_store[i, ] <- compute_statistics(X, ehat)
  }
  results[r, ] <- colMeans(stat_store)
}
# Output table
table_results <- data.frame(
  rho = rho_values,
  Ordinary = results[, 1],
  Fisher = results[, 2],
  KL = results[, 3]
)
print(table_results)

```

A.3 Empirical Power Simulation with Permutation Testing

```
# Empirical Power Table (No Averaging over rho)
library(KernSmooth)
library(TauStar)
set.seed(123)

# Regression function
g_fun <- function(x){
  sin(2 * pi * x)
}

# Kernel regression
nw_fit <- function(x, y){
  h <- dpill(x, y)
  ksmooth(
    x,
    y,
    kernel = "normal",
    bandwidth = h,
    x.points = x
  )$y
}

# Fisher information weight
fisher_weight <- function(x){
  dens <- density(x)
  fhat <- approx(dens$x, dens$y, xout = x)$y
  dx <- diff(dens$x)[1]
  deriv <- c(diff(dens$y) / dx, 0)
  fprime <- approx(dens$x[-1], deriv, xout = x)$y
  w <- abs(fprime / fhat)
  w[is.na(w)] <- 0
  return(w)
}

# KL weight
kl_weight <- function(x){
  dens <- density(x)
  fhat <- approx(dens$x, dens$y, xout = x)$y
  f0 <- dnorm(x, mean(x), sd(x))
  w <- abs(log(fhat / f0))
  w[is.na(w)] <- 0
  return(w)
}
```

```

}
# Permutation Type-I error rate
perm_test <- function(x, e, B = 500){
  obs <- TauStar(x, e)
  perm <- numeric(B)
  for(b in 1:B){
    perm[b] <- TauStar(x, sample(e))
  }
  mean(abs(perm) >= abs(obs))
}

# Simulation settings
rho_vals <- c(-0.3, -0.2, -0.1, 0.1, 0.2, 0.3)
n_vals <- c(50, 100, 200, 500)
R <- 500
alpha <- 0.05
results <- data.frame()
# Simulation loop
for(n in n_vals){
  for(rho in rho_vals){
    rej0 <- 0
    rejF <- 0
    rejKL <- 0
    for(r in 1:R){
      X <- runif(n)
      eps <- rnorm(n)
      e <- rho * X + eps
      Y <- g_fun(X) + e
      ghat <- nw_fit(X, Y)
      ehat <- Y - ghat
      # Ordinary tau*
      p0 <- perm_test(X, ehat, B = 500)
      # Fisher weighted
      wf <- fisher_weight(X)
      pF <- perm_test(wf * X, wf * ehat, B = 500)
      # KL weighted
      wk <- kl_weight(X)
      pKL <- perm_test(wk * X, wk * ehat, B = 500)
      rej0 <- rej0 + (p0 < alpha)
      rejF <- rejF + (pF < alpha)
    }
  }
}

```

```

    rejKL <- rejKL + (pKL < alpha)
  }
  power0 <- rej0 / R
  powerF <- rejF / R
  powerKL <- rejKL / R
  results <- rbind(
    results,
    data.frame(
      n = n,
      rho = rho,
      tau0 = round(power0, 3),
      tauF = round(powerF, 3),
      tauKL = round(powerKL, 3)
    )
  )
}
}
print(results)

```

A.4 Real data analysis Code

```

# Real Data Analysis: Blood Glucose Level vs Age
# Required packages
library(KernSmooth)
library(np)
# Dataset already available in the Global Environment:
# glucose_age_data
# 1. Check available objects
ls()
# 2. Check original column names
colnames(glucose_age_data)
# 3. Convert column names to safe format
names(glucose_age_data) <- make.names(names(glucose_age_data))
# Verify updated column names
colnames(glucose_age_data)
# 4. Rename glucose column for convenience
names(glucose_age_data)[
  names(glucose_age_data) == "Blood.Glucose.Level.BGL."
] <- "BGL"
# 5. Convert variables to numeric

```

```

glucose_age_data$Age <- as.numeric(glucose_age_data$Age)
glucose_age_data$BGL <- as.numeric(glucose_age_data$BGL)
# 6. Remove rows containing missing values
glucose_age_data <- na.omit(glucose_age_data)
# 7. Verify cleaned data
summary(glucose_age_data$Age)
summary(glucose_age_data$BGL)
dim(glucose_age_data)
# -----
# Scatter Plot: Blood Glucose Level vs Age
# -----
plot(
  glucose_age_data$Age,
  glucose_age_data$BGL,
  pch = 16,
  cex = 0.6,
  col = rgb(0, 0, 0, 0.5),
  xlab = "Age (years)",
  ylab = "Blood Glucose Level",
  main = "Scatter Plot of Blood Glucose Level vs Age"
)
# Save the scatter plot as PDF for LaTeX
pdf(
  "glucose_age_scatterplot.pdf",
  width = 6,
  height = 4
)
plot(
  glucose_age_data$Age,
  glucose_age_data$BGL,
  pch = 16,
  cex = 0.6,
  col = rgb(0, 0, 0, 0.5),
  xlab = "Age (years)",
  ylab = "Blood Glucose Level",
  main = "Scatter Plot of Blood Glucose Level vs Age"
)
dev.off()
# -----

```

```

# Nonparametric Kernel Regression: BGL vs Age
# -----
# Reconfirm cleaned column names
names(glucose_age_data) <- make.names(names(glucose_age_data))
names(glucose_age_data)[
  names(glucose_age_data) == "Blood.Glucose.Level.BGL."
] <- "BGL"
glucose_age_data <- na.omit(glucose_age_data)
# Kernel regression (Nadaraya-Watson estimator)
model <- npreg(
  BGL ~ Age,
  data = glucose_age_data
)
# Generate grid of age values
age_grid <- seq(
  min(glucose_age_data$Age),
  max(glucose_age_data$Age),
  length.out = 200
)
# Predicted values
fit_values <- predict(
  model,
  newdata = data.frame(Age = age_grid)
)
# Create dataset for fitted regression values
kernel_fit <- data.frame(
  Age = age_grid,
  ghat = fit_values
)
# Save fitted curve values
write.csv(
  kernel_fit,
  "kernel_regression_fit.csv",
  row.names = FALSE
)
# -----
# Scatterplot of Age vs Estimated Regression Function
# -----
# Check objects

```

```

ls()
# Check structure
str(kernel_fit)
# Check column names
names(kernel_fit)
# Reset graphics device
while(!is.null(dev.list())) dev.off()
# Extract variables
Age <- as.numeric(as.character(kernel_fit$Age))
ghat <- as.numeric(as.character(kernel_fit$ghat))
# Verify values
summary(Age)
summary(ghat)
# Remove missing values if created during conversion
ok <- complete.cases(Age, ghat)
Age <- Age[ok]
ghat <- ghat[ok]
# Scatterplot
plot(
  Age,
  ghat,
  pch = 19,
  col = "blue",
  xlab = "Age",
  ylab = "Estimated Regression Function",
  main = "Scatterplot of Age vs Estimated Function"
)
# -----
# Scatterplot with Fitted Linear Regression Line
# -----
fit_lm <- lm(
  ghat ~ Age,
  data = kernel_fit
)
plot(
  kernel_fit$Age,
  kernel_fit$ghat,
  pch = 19,
  xlab = "Age",

```

```

      ylab = "Estimated Regression Function",
      main = "Scatterplot with Fitted Regression Line"
    )
abline(
  fit_lm,
  col = "red",
  lwd = 2
)
# -----
# Kernel Regression Fit: Blood Glucose Level vs Age
# -----
# Extract variables again
Age <- as.numeric(as.character(kernel_fit$Age))
ghat <- as.numeric(as.character(kernel_fit$ghat))
# Remove missing values if necessary
ok <- complete.cases(Age, ghat)
Age <- Age[ok]
ghat <- ghat[ok]
# Bandwidth selection via direct plug-in method
bw <- dpill(Age, ghat)
# Nadaraya-Watson kernel regression estimate
fit_kernel <- locpoly(
  Age,
  ghat,
  bandwidth = bw,
  kernel = "normal",
  degree = 0
)
# Scatterplot
plot(
  Age,
  ghat,
  pch = 19,
  xlab = "Age",
  ylab = "Blood Glucose Level",
  main = "Kernel Regression Fit"
)
# Add fitted regression curve
lines(

```

```
fit_kernel$x,  
fit_kernel$y,  
col = "red",  
lwd = 2  
)
```